

Aula 06

Word Embeddings — Word2Vec e GloVe

PROCESSAMENTO DE LINGUAGEM NATURAL - IBM3130

Prof. Anderson Vanin

Introdução

Nas últimas aulas aprendemos a representar textos como vetores numéricos. O *Bag of Words* conta palavras. O *TF-IDF* pondera pela raridade. Os *N-gramas* capturam sequências. Mas todas essas representações têm uma limitação fundamental em comum:

Elas não entendem o significado das palavras.

Introdução

Para o BoW, '**carro**' e '**automóvel**' são dois tokens completamente diferentes — como se não tivessem nenhuma relação. O vetor de '**rei**' não tem nenhuma proximidade com o vetor de '**rainha**'. E '**bom**' não está perto de '**excelente**' — são apenas colunas diferentes na mesma matriz esparsa.

Introdução

Os **Word Embeddings** resolvem exatamente esse problema. Eles representam cada palavra como um vetor denso em um espaço de alta dimensão, onde **palavras com significados parecidos ficam próximas**.

Esse é um dos avanços mais importantes da história do PLN — e é a **base conceitual dos modelos modernos como o BERT e o GPT**.

PARTE 1

O Problema das Representações Esparsas

1.1 Revisando o Bag of Words

No BoW, cada palavra do vocabulário é uma dimensão do vetor. Para um vocabulário de 10.000 palavras, cada texto vira um vetor com 10.000 posições — **onde a maioria é zero.**

1.1 Revisando o Bag of Words

Vocabulário: [bom, carro, excelente, gato, ótimo, péssimo, ...]

↑ 10.000 colunas

"o carro é bom" → [1, 1, 0, 0, 0, 0, ...]

"o carro é ótimo" → [0, 1, 0, 0, 1, 0, ...]

Esses dois vetores têm pouquíssima sobreposição — mas as frases têm o mesmo sentido. **O BoW não consegue enxergar isso.**

1.2 Os três problemas que os embeddings resolvem

Problema 1 — Sem semântica

No BoW, '**bom**' e '**excelente**' são dimensões completamente independentes. Um modelo treinado com exemplos contendo '**bom**' não aprende nada sobre '**excelente**' — mesmo que as duas palavras tenham o mesmo significado.

1.2 Os três problemas que os embeddings resolvem

BoW:

'bom' → [0, 0, 0, 1, 0, 0, 0, ...] ← dimensão 3

'excelente' → [0, 0, 0, 0, 0, 1, 0, ...] ← dimensão 5

Distância entre elas: **MÁXIMA** (não têm nada em comum no vetor)

Embedding:

'bom' → [0.82, -0.11, 0.45, ...]

'excelente' → [0.79, -0.08, 0.51, ...]

Distância entre elas: **MÍNIMA** (vetores quase idênticos)

1.2 Os três problemas que os embeddings resolvem

Problema 2 — Dimensionalidade

Com 100.000 palavras no vocabulário, cada vetor BoW tem **100.000 dimensões**. Em espaços de alta dimensão, todos os pontos ficam igualmente distantes — a noção de "*proximidade*" perde o sentido.

Com embeddings, usamos tipicamente **100 a 300 dimensões** — muito mais compacto e matematicamente tratável.

1.2 Os três problemas que os embeddings resolvem

Problema 3 — Esparsidade

99% das entradas do vetor BoW são zero. Isso desperdiça memória, torna o treinamento lento e dificulta o aprendizado.

Embeddings são densos — todos os valores são números reais significativos.

1.3 A ideia central dos embeddings

A hipótese que fundamenta os word embeddings é conhecida como **hipótese distribucional**, formulada pelo linguista John Rupert Firth em 1957:

"Você conhece uma palavra pelo contexto em que ela aparece."

1.3 A ideia central dos embeddings

Palavras que aparecem nos mesmos contextos tendem a ter significados parecidos. '**cachorro**' e '**gato**' aparecem em contextos similares:

"meu ____ latiu / miou"

"o ____ dorme no sofá"

"dei comida para o ____"

Portanto, '**cachorro**' e '**gato**' devem ter vetores próximos. **Essa hipótese é o que o Word2Vec operacionaliza de forma computacional.**

PARTE 2

Word2Vec

2.1 O que é Word2Vec?

Word2Vec é um conjunto de técnicas desenvolvido por Tomas Mikolov e equipe no Google em 2013. **Ele usa uma rede neural rasa (uma única camada oculta) para aprender representações densas de palavras a partir de grandes corpora.**

2.1 O que é Word2Vec?

O resultado são vetores numéricos onde:

- Palavras semelhantes ficam próximas no espaço vetorial
- Relações semânticas se traduzem em direções geométricas

Word2Vec tem duas arquiteturas principais: **CBOW** e **Skip-gram**.

2.2 Arquitetura CBOW — *Continuous Bag of Words*

Dadas palavras ao redor, prever a palavra do meio.

Janela de contexto = 2 palavras de cada lado

Frase: "o modelo neural aprende com os dados"

Entrada (contexto): ["o", "modelo", "aprende", "com"]

Saída (alvo): "neural"

O modelo aprende os pesos da rede para fazer essa previsão. **Os pesos da camada oculta são os embeddings.**

2.2 Arquitetura CBOW — Continuous Bag of Words

Visualizando o CBOW

[o]	└				
[modelo]	└┐→	[camada oculta 300 dims]	→	$P(\text{"neural"}) = 0.72$	
[aprende]	└			$P(\text{"rápido"}) = 0.05$	
[com]	└			$P(\text{"lento"}) = 0.01$	
				...	

2.2 Arquitetura CBOW — Continuous Bag of Words

Características do CBOW

- Mais rápido para treinar.
- Funciona melhor com corpus grande.
- Suaviza sobre o contexto — ignora detalhes de palavras raras.

2.3 Arquitetura Skip-gram

Dada uma palavra, prever as palavras ao redor.

Frase: "o modelo neural aprende com os dados"

Entrada (palavra central): "neural"

Saída (contexto): ["o", "modelo", "aprende", "com"]

É o processo inverso do CBOW.

2.3 Arquitetura Skip-gram

Visualizando o Skip-gram

$$\rightarrow P(\text{"o"}) = 0.15$$

$$\rightarrow P(\text{"modelo"}) = 0.68$$

`["neural"]` $\rightarrow$ `[camada oculta]`

$$\rightarrow P(\text{"aprende"}) = 0.71$$

$$\rightarrow P(\text{"com"}) = 0.12$$

2.3 Arquitetura Skip-gram

Características do Skip-gram

- Mais lento para treinar.
- Funciona melhor com corpus pequeno.
- Captura melhor palavras raras e infrequentes.
- Preferido na maioria dos casos práticos.

2.4 O que o Word2Vec aprende?

O Word2Vec aprende que **palavras usadas em contextos similares devem ter vetores similares**. Vejamos alguns exemplos do que emerge espontaneamente desse aprendizado:

2.4 O que o Word2Vec aprende?

Similaridade semântica

'rei' → [0.82, 0.31, -0.14, 0.56, ...]

'rainha' → [0.79, 0.28, -0.11, 0.54, ...] ← muito próximos!

'cachorro' → [0.43, -0.22, 0.67, 0.11, ...]

'gato' → [0.41, -0.19, 0.65, 0.13, ...] ← muito próximos!

'cachorro' vs 'computador' → completamente distantes

2.4 O que o Word2Vec aprende?

Analogias vetoriais

O resultado mais famoso do Word2Vec: relações semânticas se traduzem em **operações aritméticas com os vetores**.

`rei - homem + mulher ≈ rainha`

Isso significa: "*a relação entre rei e homem é a mesma que entre rainha e mulher.*"

2.4 O que o Word2Vec aprende?

Paris - França + Itália $\approx$ Roma

Brasil - português + espanhol $\approx$ Argentina

médico - homem + mulher $\approx$ médica

Essas analogias emergem automaticamente do treinamento — ninguém programou essas relações explicitamente. O modelo as aprendeu a partir dos padrões de co-ocorrência no texto.

2.5 Treinando Word2Vec com Gensim

O **Gensim** é a biblioteca mais usada para *Word2Vec* em Python.

2.5 Treinando Word2Vec com Gensim

```
# Instalar no Colab
```

```
!pip install gensim -q
```

```
from gensim.models import Word2Vec
```

```
# Corpus simples – lista de listas de tokens
```

```
corpus = [  
    ["o", "gato", "dorme", "no", "sofá"],  
    ["o", "cachorro", "corre", "no", "parque"],  
    ["o", "gato", "come", "o", "rato"],  
    ["o", "cachorro", "late", "muito"],  
    ["o", "gato", "e", "o", "cachorro", "brincam"],  
    ["o", "rato", "corre", "do", "gato"],  
    ["o", "cachorro", "dorme", "no", "chão"],  
    ["o", "gato", "late", "igual", "ao", "cachorro"],  
]
```

2.5 Treinando Word2Vec com Gensim

```
# Treinar o modelo Word2Vec
```

```
modelo = Word2Vec(  
    sentences = corpus,      # corpus – lista de listas de tokens  
    vector_size = 10,       # dimensões do embedding (baixo para exemplo didático)  
    window = 2,             # janela de contexto – palavras de cada lado  
    min_count = 1,          # mínimo de ocorrências para incluir a palavra  
    sg = 1,                 # 1 = Skip-gram, 0 = CBOW  
    epochs = 100,           # número de passagens pelo corpus  
    seed = 42               # para reprodutibilidade  
)
```

```
print("Modelo treinado!")
```

```
print("Vocabulário:", list(modelo.wv.key_to_index.keys()))
```

```
print("Dimensões do embedding:", modelo.wv.vector_size)
```

2.5 Treinando Word2Vec com Gensim

Modelo treinado!

Vocabulário: ['o', 'cachorro', 'gato', 'no', 'late', 'rato', 'corre', 'dorme', 'ao', 'igual', 'chão', 'do', 'brincam', 'e', 'muito', 'come', 'parque', 'sofá']

Dimensões do embedding: 10

2.5 Treinando Word2Vec com Gensim

```
# Ver o vetor de uma palavra
```

```
vetor_gato = modelo.wv["gato"]
```

```
print("Vetor de 'gato':")
```

```
print(vetor_gato)
```

```
print("Formato:", vetor_gato.shape)
```

Vetor de 'gato':

```
[ 0.00167185 -0.02553395 -0.06408041  0.08661403  0.06134353  0.0321628  
 -0.0235205   0.06517179  0.00921127 -0.01275574]
```

Formato: (10,)

```
# Ver o vetor de outra palavra
```

```
vetor_cachorro = modelo.wv["cachorro"]
```

```
print("\nVetor de 'cachorro':")
```

```
print(vetor_cachorro)
```

Vetor de 'cachorro':

```
[ 0.00665445  0.09489229  0.0469921   0.05237921  0.04736762  0.05850109  
 0.00079376 -0.0743202   0.06764559 -0.01045547]
```

2.5 Treinando Word2Vec com Gensim

```
# Encontrar palavras mais parecidas com 'gato'
similares_gato = modelo.wv.most_similar("gato", topn=3)

print("Palavras mais similares a 'gato':")
for palavra, similaridade in similares_gato:
    print(f" {palavra:<15} similaridade: {similaridade:.4f}")

# Fazer o mesmo para 'cachorro'
similares_cachorro = modelo.wv.most_similar("cachorro", topn=3)

print("\nPalavras mais similares a 'cachorro':")
for palavra, similaridade in similares_cachorro:
    print(f" {palavra:<15} similaridade: {similaridade:.4f}")
```

```
Palavras mais similares a 'gato':

ao          similaridade: 0.4558
no          similaridade: 0.4203
muito       similaridade: 0.3411
```

```
Palavras mais similares a 'cachorro':

igual       similaridade: 0.3181
do          similaridade: 0.3014
ao          similaridade: 0.2794
```


PARTE 3

Similaridade de Cosseno

3.1 Como medir a proximidade entre vetores?

Dado que cada palavra é um vetor, precisamos de uma forma de medir o quão próximos dois vetores estão. A medida mais usada em PLN é a **similaridade de cosseno**.

3.2 A ideia geométrica

A similaridade de cosseno não mede a distância entre dois pontos, mas o ângulo entre os dois vetores:

Vetores paralelos (ângulo = 0°) → cosseno = 1,0 → máxima similaridade

Vetores perpendiculares (ângulo = 90°) → cosseno = 0,0 → sem relação

Vetores opostos (ângulo = 180°) → cosseno = -1,0 → significados opostos

Isso é importante porque **palavras com contextos similares tendem a apontar para a mesma direção no espaço vetorial** — mesmo que tenham magnitudes diferentes.

3.3 A fórmula

$$\cos(A, B) = \frac{A \cdot B}{|A| \times |B|}$$

Onde:

$A \cdot B$ = produto escalar (soma dos produtos elemento a elemento)

$|A|$ = normalizado do vetor A (raiz quadrada da soma dos quadrados)

$|B|$ = normalizado do vetor B

3.4 Exemplo numérico completo

```
import math
```

```
# Vetores simplificados de duas palavras (apenas 3 dimensões para facilitar)
```

```
vetor_gato      = [0.8, 0.3, 0.5]
```

```
vetor_cachorro = [0.7, 0.4, 0.6]
```

```
vetor_carro     = [0.1, 0.9, 0.2]
```

3.4 Exemplo numérico completo

```
def similaridade_cosseno(v1, v2):

    # Passo 1: produto escalar ( $A \cdot B$ )
    produto = 0
    for i in range(len(v1)):
        produto = produto + v1[i] * v2[i]

    # Passo 2: norma do vetor v1 ( $|A|$ )
    soma_quadrados_v1 = 0
    for x in v1:
        soma_quadrados_v1 = soma_quadrados_v1 + x * x
    norma_v1 = math.sqrt(soma_quadrados_v1)

    # Passo 3: norma do vetor v2 ( $|B|$ )
    soma_quadrados_v2 = 0
    for x in v2:
        soma_quadrados_v2 = soma_quadrados_v2 + x * x
    norma_v2 = math.sqrt(soma_quadrados_v2)

    # Passo 4: dividir
    return produto / (norma_v1 * norma_v2)
```


3.4 Exemplo numérico completo

```
# Calcular similaridades

sim_gato_cachorro = similaridade_cosseno(vetor_gato, vetor_cachorro)
sim_gato_carro    = similaridade_cosseno(vetor_gato, vetor_carro)

print("=== Similaridade de Cosseno ===")
print()
print(f"  'gato' vs 'cachorro' = {sim_gato_cachorro:.4f}")
print(f"  'gato' vs 'carro'    = {sim_gato_carro:.4f}")
print()

if sim_gato_cachorro > sim_gato_carro:
    print("  → 'cachorro' é mais similar a 'gato' do que 'carro'")
    print("    (faz sentido – ambos são animais de estimação!)")
```

```
=== Similaridade de Cosseno ===

'gato' vs 'cachorro' = 0.9850
'gato' vs 'carro'    = 0.4902
```

3.5 Similaridade com Gensim

O Gensim calcula a similaridade de cosseno automaticamente:

```
# Similaridade entre duas palavras usando Gensim
```

```
sim = modelo.wv.similarity("gato", "cachorro")
```

```
Similaridade('gato', 'cachorro') = -0.0080
```

```
print(f"Similaridade('gato', 'cachorro') = {sim:.4f}")
```

```
sim2 = modelo.wv.similarity("gato", "rato")
```

```
Similaridade('gato', 'rato') = -0.1988
```

```
print(f"Similaridade('gato', 'rato') = {sim2:.4f}")
```

```
sim3 = modelo.wv.similarity("gato", "sofá")
```

```
Similaridade('gato', 'sofá') = -0.0066
```

```
print(f"Similaridade('gato', 'sofá') = {sim3:.4f}")
```


3.5 Similaridade com Gensim

Qual par é mais similar?

```
pares = [  
    ("gato", "cachorro"),  
    ("gato", "rato"),  
    ("gato", "sofá"),  
    ("cachorro", "rato"),  
]
```

3.5 Similaridade com Gensim

```
print("\nRanking de similaridade:")
resultados = []
for p1, p2 in pares:
    sim = modelo.wv.similarity(p1, p2)
    resultados.append((p1, p2, sim))

resultados.sort(key=lambda x: x[2], reverse=True)

posicao = 1
for p1, p2, sim in resultados:
    print(f" {posicao}. '{p1}' vs '{p2}': {sim:.4f}")
    posicao = posicao + 1
```

Ranking de similaridade:

1. 'gato' vs 'sofá': -0.0066
2. 'gato' vs 'cachorro': -0.0080
3. 'cachorro' vs 'rato': -0.1555
4. 'gato' vs 'rato': -0.1988

+1.0 → vetores apontam na mesma direção → muito similares
0.0 → vetores perpendiculares → sem relação
-1.0 → vetores apontam em direções opostas → significados opostos

PARTE 4

Analogias Vetoriais

4.1 O que são analogias vetoriais?

Uma das descobertas mais surpreendentes do Word2Vec foi que relações semânticas se traduzem em direções geométricas consistentes. Se o modelo aprendeu que '**rei**' é para '**homem**' assim como '**rainha**' é para '**mulher**', então geometricamente:

$$\text{vetor}(\text{"rei"}) - \text{vetor}(\text{"homem"}) + \text{vetor}(\text{"mulher"}) \approx \text{vetor}(\text{"rainha"})$$

Isso funciona porque o modelo aprendeu, implicitamente, que existe uma direção no espaço vetorial que representa o conceito de "*gênero feminino vs. masculino*".

4.2 Como funciona na prática

Usando Gensim – positive = palavras a somar, negative = palavras a subtrair

```
resultado = modelo.wv.most_similar(
```

```
    positive=["rainha", "homem"],
```

```
    negative=["rei"],
```

```
    topn=3
```

```
)
```

**Não usar este código ainda!
Será utilizado na parte 5
desta aula!**

Equivale a: rainha - rei + homem $\approx$?

```
print("rainha - rei + homem  $\approx$  ?")
```

```
for palavra, similaridade in resultado:
```

```
    print(f" {palavra} ({similaridade:.4f})")
```

4.3 Exemplos famosos de analogias

Esses exemplos foram demonstrados no corpus Google News (100 bilhões de palavras):

Relações de gênero

rei - homem + mulher $\approx$ rainha

ator - homem + mulher $\approx$ atriz

Capitais de países

Paris - França + Itália $\approx$ Roma

Paris - França + Brasil $\approx$ Brasília

Formas verbais

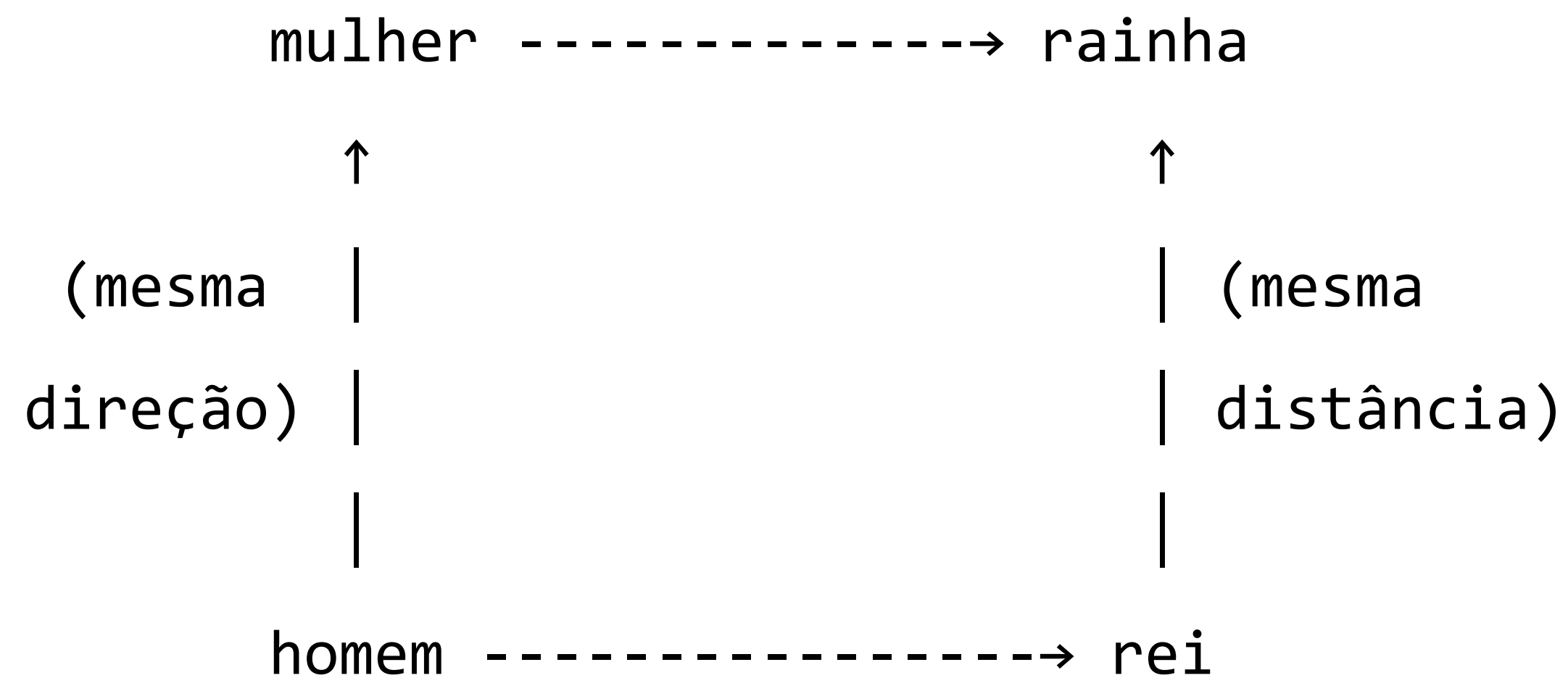
correr - correndo + andando $\approx$ andar

Comparativos

bom - melhor + pior $\approx$ ruim

4.4 Por que isso acontece?

Imagine o espaço vetorial como um mapa. Cada palavra é um ponto nesse mapa. Se você "*caminhar*" do ponto '**homem**' ao ponto '**mulher**', percorrerá uma distância em uma direção específica. Essa mesma distância e direção, aplicada a partir do ponto '**rei**', chega perto do ponto '**rainha**'.



PARTE 5

Embeddings Pré-treinados

5.1 O problema do corpus pequeno

O Word2Vec precisa de muito texto para aprender boas representações. **Com corpus pequeno, os vetores ficam ruins** — o modelo não vê contextos suficientes.

A solução: usar embeddings pré-treinados em corpus massivos.

5.2 Embeddings disponíveis publicamente

Modelo	Corpus de treino	Dimensões	Tamanho
Word2Vec (Google News)	100 bilhões de palavras (inglês)	300	1,5 GB
GloVe (Wikipedia + Gigaword)	6 bilhões de tokens (inglês)	50–300	822 MB
fastText (Wikipedia)	Múltiplos idiomas	300	~6 GB
NILC/USP (português)	Corpus brasileiro	50–600	Variado

5.3 Usando embeddings pré-treinados com Gensim

```
import gensim.downloader as api

# Baixar embeddings pré-treinados (pode demorar – arquivo grande)
# 'glove-wiki-gigaword-50' é o menor disponível: 50 dimensões
modelo_pt = api.load("glove-wiki-gigaword-50")

print("Modelo carregado!")
print("Vocabulário:", len(modelo_pt.key_to_index), "palavras")
print("Dimensões:", modelo_pt.vector_size)

# Testar similaridade
print("\nSimilaridade 'king' vs 'queen':", modelo_pt.similarity("king", "queen"))
print("Similaridade 'cat' vs 'dog':    ", modelo_pt.similarity("cat", "dog"))
print("Similaridade 'cat' vs 'car':    ", modelo_pt.similarity("cat", "car"))
```

```
Modelo carregado!
Vocabulário: 400000 palavras
Dimensões: 50
```

```
Similaridade 'king' vs 'queen': 0.7839043
Similaridade 'cat' vs 'dog':    0.9218005
Similaridade 'cat' vs 'car':    0.36382526
```


5.3 Usando embeddings pré-treinados com Gensim

```
# Analogia: king - man + woman = ?
resultado = modelo_pt.most_similar(
    positive=["king", "woman"],
    negative=["man"],
    topn=3
)
print("\nking - man + woman ≈")
for palavra, sim in resultado:
    print(f"  {palavra} ({sim:.4f})")
```

```
king - man + woman ≈
  queen (0.8524)
  throne (0.7664)
  prince (0.7592)
```

5.4 Embeddings para português — NILC/USP

Para projetos em português, o **NILC (USP)** disponibiliza embeddings treinados em corpus brasileiro. Os embeddings do NILC agora estão disponíveis no Hugging Face, que é muito mais estável que o servidor da USP.

```
# Instalar biblioteca do Hugging Face
!pip install huggingface_hub safetensors -q

from huggingface_hub import hf_hub_download
from safetensors.numpy import load_file
from gensim.models import KeyedVectors
import numpy as np
```

5.4 Embeddings para português — NILC/USP

```
# Baixar embeddings e vocabulário

# repo_id : nilc-nlp/word2vec-skip-gram-50d

path_emb = hf_hub_download(repo_id="nilc-nlp/word2vec-skip-gram-50d", filename="embeddings.safetensors")
path_vocab = hf_hub_download(repo_id="nilc-nlp/word2vec-skip-gram-50d", filename="vocab.txt")


# Carregar vetores e vocabulário

data = load_file(path_emb)
vetores = data["embeddings"]


with open(path_vocab, encoding="utf-8") as f:
    vocab = [linha.strip() for linha in f]


print(f"Vocabulário: {len(vocab)} palavras")
print(f"Dimensões: {vetores.shape}")
```

Vocabulário: 929606 palavras
Dimensões: (929606, 50)

5.4 Embeddings para português — NILC/USP

```
# Criar o objeto KeyedVectors com os dados carregados
modelo_pt = KeyedVectors(vector_size=vetores.shape[1])
modelo_pt.add_vectors(vocab, vetores)

print("Modelo montado!")
print(f"Total de palavras: {len(modelo_pt.key_to_index)}")
print(f"Dimensões:          {modelo_pt.vector_size}")
```

```
Modelo montado!
Total de palavras: 929603
Dimensões:          50
```


5.4 Embeddings para português — NILC/USP

```
# Confirmar que as palavras que vamos usar existem no modelo
```

```
palavras_teste = ["rei", "rainha", "homem", "mulher",  
                  "gato", "cachorro", "bom", "excelente"]
```

```
print("Palavras no vocabulário:")
```

```
for palavra in palavras_teste:
```

```
    status = "✓" if palavra in modelo_pt else "X"
```

```
    print(f" {status} '{palavra}')
```

```
print("Palavras similares a 'inteligência' em português:")
```

```
for palavra, sim in modelo_pt.most_similar("inteligência", topn=5):
```

```
    print(f" {palavra:<20} {sim:.4f}")
```

Palavras no vocabulário:

- ✓ 'rei'
- ✓ 'rainha'
- ✓ 'homem'
- ✓ 'mulher'
- ✓ 'gato'
- ✓ 'cachorro'
- ✓ 'bom'
- ✓ 'excelente'

Palavras similares a 'inteligência' em português:

cibernética	0.7472
contraespionagem	0.6953
confiabilidade	0.6913
tecnologia	0.6823
contra-espionagem	0.6799

5.4 Embeddings para português — NILC/USP

```
# Comparar pares de palavras
pares = [
    ("gato", "cachorro"),
    ("rei", "rainha"),
    ("bom", "excelente"),
    ("bom", "péssimo"),
    ("gato", "computador"),
]

print("Similaridade de cosseno entre pares:")
print()
for p1, p2 in pares:
    sim = modelo_pt.similarity(p1, p2)
    print(f" '{p1}' vs '{p2}': {sim:.4f}")
```

Similaridade de cosseno entre pares:

```
'gato' vs 'cachorro': 0.8801
'rei' vs 'rainha': 0.6748
'bom' vs 'excelente': 0.5178
'bom' vs 'péssimo': 0.7596
'gato' vs 'computador': 0.3555
```

5.4 Embeddings para português — NILC/USP

```
# Agora sim – analogias funcionando em português!
```

```
# Analogias vetoriais em português
```

```
analogias = [  
    {  
        "positive": ["rainha", "homem"],  
        "negative": ["rei"],  
        "descricao": "rainha - rei + homem ≈ ?"  
    },  
    {  
        "positive": ["paris", "brasil"],  
        "negative": ["frança"],  
        "descricao": "paris - frança + brasil ≈ ?"  
    },  
    {  
        "positive": ["atriz", "homem"],  
        "negative": ["mulher"],  
        "descricao": "atriz - mulher + homem ≈ ?"  
    },  
]
```

5.4 Embeddings para português — NILC/USP

```
for a in analogias:
    print(a["descricao"])
    resultado = modelo_pt.most_similar(
        positive = a["positive"],
        negative = a["negative"],
        topn      = 3
    )
    for palavra, sim in resultado:
        print(f"  {palavra:<20} {sim:.4f}")
    print()
```

```
rainha - rei + homem ≈ ?
  mocinha           0.7797
  menina            0.7757
  moça              0.7737

paris - França + Brasil ≈ ?
  carnatal          0.7339
  playcenter        0.7055
  tedx              0.7039

atriz - mulher + homem ≈ ?
  ator              0.8049
  músico            0.7981
  ilusionista       0.7782
```


PARTE 6

GloVe

6.1 O que é GloVe?

GloVe (*Global Vectors for Word Representation*) foi desenvolvido por Jeffrey Pennington, Richard Socher e Christopher Manning em Stanford em 2014.

Enquanto o **Word2Vec** aprende **embeddings** a partir de janelas de contexto locais (palavras vizinhas), o **GloVe** usa **estatísticas globais de co-ocorrência** — conta quantas vezes cada par de palavras aparece juntas em todo o corpus.

6.2 A intuição do GloVe

O GloVe parte de uma pergunta: o que a proporção entre as probabilidades de co-ocorrência de palavras nos diz sobre seus significados?

Considere as palavras: "gelo", "água", "sólido", "gás"

$P(\text{"sólido"} \mid \text{"gelo"})$ é ALTA – gelo é sólido

$P(\text{"sólido"} \mid \text{"água"})$ é BAIXA – água não é sólida (em geral)

$P(\text{"gás"} \mid \text{"gelo"})$ é BAIXA

$P(\text{"gás"} \mid \text{"água"})$ é ALTA – água pode virar gás (vapor)

A razão:

$P(\text{"sólido"} \mid \text{"gelo"}) / P(\text{"sólido"} \mid \text{"água"}) \rightarrow \text{valor ALTO}$

$P(\text{"gás"} \mid \text{"gelo"}) / P(\text{"gás"} \mid \text{"água"}) \rightarrow \text{valor BAIXO}$

Essas proporções **capturam** relações semânticas de forma mais robusta do que probabilidades individuais.

6.4 Usando GloVe com Gensim

Baixar o Glove em: <https://huggingface.co/stanfordnlp/glove/resolve/main/glove.6B.zip>

```
# O Gensim pode carregar embeddings no formato GloVe
```

```
# Após baixar o arquivo glove.6B.50d.txt
```

```
from gensim.scripts.glove2word2vec import glove2word2vec
```

```
from gensim.models import KeyedVectors
```

```
# Converter formato GloVe para formato Word2Vec
```

```
glove2word2vec("glove.6B.50d.txt", "glove_w2v.txt")
```

```
# Carregar
```

```
glove = KeyedVectors.load_word2vec_format("glove_w2v.txt", binary=False)
```

```
# Usar normalmente
```

```
print(glove.most_similar("computer", topn=5))
```

```
print(glove.similarity("good", "excellent"))
```

```
[('computers', 0.9165045022964478), ('software', 0.8814992904663086), ('technology', 0.852556049823761),
```

```
0.8061553
```

PARTE 7

Visualizando Embeddings com PCA

7.1 O problema de visualizar alta dimensão

Um embedding típico tem 100 a 300 dimensões. Não é possível visualizar um espaço de 300 dimensões diretamente.

A solução: usar **PCA** (*Principal Component Analysis*) para comprimir os vetores para apenas 2 dimensões, preservando ao máximo as distâncias originais.

7.2 O que é PCA?

PCA encontra as duas direções no espaço de alta dimensão que mais "explicam" a variação dos dados — e projeta todos os pontos nessas direções.

300 dimensões → PCA → 2 dimensões → gráfico de dispersão

É como fotografar um objeto 3D: você perde uma dimensão, mas consegue ver a estrutura geral.

7.3 Visualizando Word2Vec com PCA

```
import math

from gensim.models import Word2Vec

# Corpus sobre tecnologia e animais – para ver grupos distintos
corpus = [
    ["inteligência", "artificial", "aprende", "dados"],
    ["rede", "neural", "processa", "texto"],
    ["algoritmo", "machine", "learning", "treina"],
    ["modelo", "linguagem", "texto", "palavras"],
    ["computador", "processa", "dados", "rápido"],
    ["gato", "dorme", "sofá", "ronrona"],
    ["cachorro", "late", "corre", "brinca"],
    ["pássaro", "voa", "canta", "ninho"],
    ["peixe", "nada", "água", "aquário"],
    ["coelho", "pula", "come", "cenoura"],
]
```


7.3 Visualizando Word2Vec com PCA

```
# Treinar Word2Vec
modelo = Word2Vec(
    sentences      = corpus,
    vector_size    = 20,
    window         = 2,
    min_count      = 1,
    sg             = 1,
    epochs         = 200,
    seed           = 42
)
```

7.3 Visualizando Word2Vec com PCA

```
# Palavras que vamos visualizar

palavras_tecnologia = ["inteligência", "rede", "algoritmo", "modelo", "computador"]

palavras_animais = ["gato", "cachorro", "pássaro", "peixe", "coelho"]

palavras_todas = palavras_tecnologia + palavras_animais
```

```
# Extrair vetores

vetores = []

for palavra in palavras_todas:

    if palavra in modelo.wv:

        vetores.append(modelo.wv[palavra].tolist())

print(f"Total de vetores: {len(vetores)}")

print(f"Dimensões originais: {len(vetores[0])}")
```

Total de vetores: 10
Dimensões originais: 20

```
Palavra: "inteligência" [Tecnologia]
Dimensões: 20
Vetor: [-0.035, 0.0443, 0.033, -0.0368, 0.0291, -0.0361, 0.0188, -0.0412, -0.0008, 0.0157,

Palavra: "rede" [Tecnologia]
Dimensões: 20
Vetor: [-0.0318, 0.0237, -0.0475, -0.0291, 0.0106, 0.0199, -0.0014, 0.0363, 0.0422, -0.037

Palavra: "algoritmo" [Tecnologia]
Dimensões: 20
Vetor: [-0.0055, -0.0008, -0.0356, -0.0078, 0.0124, -0.0263, -0.0192, -0.0141, 0.0016, -0.
```

7.3 Visualizando Word2Vec com PCA

```
# Redução de dimensionalidade manual com PCA simplificado
```

```
# (sem sklearn – usando apenas operações básicas)
```

```
# Passo 1: Centralizar os vetores (subtrair a média)
```

```
n_palavras = len(vetores)
```

```
n_dims      = len(vetores[0])
```

```
# Calcular a média de cada dimensão
```

```
medias = []
```

```
for d in range(n_dims):
```

```
    soma = 0
```

```
    for v in vetores:
```

```
        soma = soma + v[d]
```

```
    medias.append(soma / n_palavras)
```


7.3 Visualizando Word2Vec com PCA

```
# Subtrair a média
vetores_centralizados = []

for v in vetores:
    v_cent = []
    for d in range(n_dims):
        v_cent.append(v[d] - medias[d])
    vetores_centralizados.append(v_cent)

print("Vetores centralizados (primeiros 3 valores do vetor 0):")
print(vetores_centralizados[0][:3])
```

Usando sklearn apenas para o PCA

7.3 Visualizando Word2Vec com PCA

```
import matplotlib.pyplot as plt

# Usando sklearn apenas para o PCA (não há forma simples sem ele)
from sklearn.decomposition import PCA

import numpy as np

# Converter para array numpy
X = np.array(vetores)

# Aplicar PCA – reduzir para 2 dimensões
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)
```

7.3 Visualizando Word2Vec com PCA

```
# Plotar
```

```
fig, ax = plt.subplots(figsize=(10, 7))
```

```
# Plotar tecnologia em azul
```

```
for i in range(len(palavras_tecnologia)):
```

```
    ax.scatter(X_2d[i, 0], X_2d[i, 1], color='#2E75B6', s=100)
```

```
    ax.annotate(palavras_todas[i],
```

```
                xy=(X_2d[i, 0], X_2d[i, 1]),
```

```
                xytext=(5, 5), textcoords='offset points',
```

```
                fontsize=11, color='#2E75B6')
```

7.3 Visualizando Word2Vec com PCA

```
# Plotar animais em verde
for i in range(len(palavras_tecnologia), len(palavras_todas)):
    ax.scatter(X_2d[i, 0], X_2d[i, 1], color='#1A5E3A', s=100)
    ax.annotate(palavras_todas[i],
                xy=(X_2d[i, 0], X_2d[i, 1]),
                xytext=(5, 5), textcoords='offset points',
                fontsize=11, color='#1A5E3A')

ax.set_title('Word Embeddings visualizados em 2D (PCA)',
            fontsize=13, fontweight='bold')
ax.set_xlabel('Componente Principal 1')
ax.set_ylabel('Componente Principal 2')
ax.axhline(0, color='gray', linewidth=0.5, linestyle='--')
ax.axvline(0, color='gray', linewidth=0.5, linestyle='--')
ax.grid(True, alpha=0.3)
```


7.3 Visualizando Word2Vec com PCA

```
# Legenda manual

from matplotlib.patches import Patch

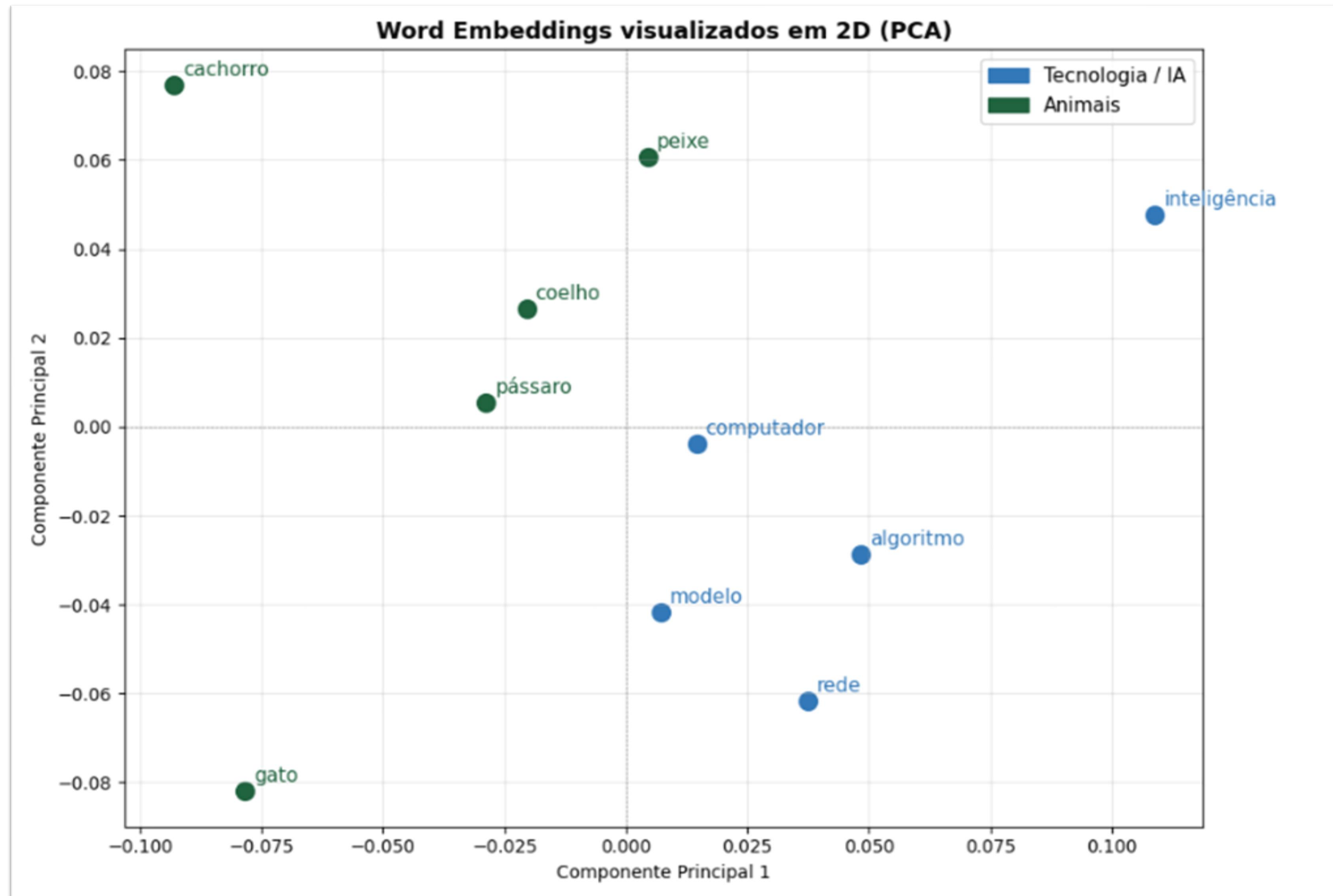
legenda = [
    Patch(color='#2E75B6', label='Tecnologia / IA'),
    Patch(color='#1A5E3A', label='Animais'),
]

ax.legend(handles=legenda, fontsize=11)

plt.tight_layout()
plt.show()

print("Observe como palavras do mesmo campo semântico")
print("ficam agrupadas no gráfico!")
```

7.3 Visualizando Word2Vec com PCA



7.4 O que esperar do gráfico

Se os embeddings foram bem aprendidos, você deve ver:

- Palavras de tecnologia ('rede', 'algoritmo', 'modelo') **agrupadas**.
- Palavras de animais ('gato', 'cachorro', 'peixe') **em outro grupo**.
- Os dois grupos separados no espaço 2D.

Com um corpus tão pequeno (10 frases), os grupos podem não estar perfeitos — mas com um corpus maior, a separação fica muito clara.

PARTE 8

Word Embeddings em PLN Moderno

8.1 Embeddings estáticos vs. contextualizados

Os embeddings que vimos até agora são estáticos: cada palavra tem um único vetor, independentemente do contexto em que aparece.

"banco" em "fui ao banco sacar dinheiro" → mesmo vetor

"banco" em "sentei no banco da praça" → mesmo vetor ← problema!

8.1 Embeddings estáticos vs. contextualizados

Os modelos modernos usam embeddings contextualizados: o vetor de uma palavra muda dependendo da frase em que ela aparece.

Tipo	Exemplos	Característica
Estático	Word2Vec, GloVe, fastText	Um vetor por palavra
Contextualizado	BERT, GPT, RoBERTa	Vetor muda com o contexto

8.2 A evolução dos embeddings

Representação one-hot (BoW)

↓ sem semântica

Word2Vec / GloVe (2013–2014)

↓ semântica estática

ELMo (2018) → embeddings com contexto (LSTMs)

↓ ainda sequencial

BERT (2018) → contexto bidirecional completo (Transformers)

↓ ainda maior

GPT-3, GPT-4, Claude (2020+) → LLMs com bilhões de parâmetros

8.3 Aplicações práticas de embeddings

```
# Aplicação 1: busca semântica
```

```
# Encontrar documentos semanticamente similares à query
```

```
query    = "inteligência artificial aprende"
```

```
docs     = ["machine learning processa dados",  
            "gato dorme no sofá",  
            "rede neural treina modelo"]
```


8.3 Aplicações práticas de embeddings

```
# Vetorizar query e documentos com word2vec
# (média dos vetores das palavras)
def media_vetores(tokens, modelo):
    vetores_validos = []
    for token in tokens:
        if token in modelo.wv:
            vetores_validos.append(modelo.wv[token])
    if not vetores_validos:
        return None
    # Calcular média manualmente
    n = len(vetores_validos)
    dims = len(vetores_validos[0])
    media = []
    for d in range(dims):
        soma = 0
        for v in vetores_validos:
            soma = soma + v[d]
        media.append(soma / n)
    return media
```

8.3 Aplicações práticas de embeddings

```
from nltk.tokenize import word_tokenize

tokens_query = word_tokenize(query.lower(), language='portuguese')
vetor_query = media_vetores(tokens_query, modelo)

print("Busca semântica para:", query)
print()
```

8.3 Aplicações práticas de embeddings

```
resultados = []  
  
for doc in docs:  
    tokens_doc = word_tokenize(doc.lower(), language='portuguese')  
    vetor_doc = media_vetores(tokens_doc, modelo)  
    if vetor_doc and vetor_query:  
        sim = similaridade_cosseno(vetor_query, vetor_doc)  
        resultados.append((doc, sim))  
  
resultados.sort(key=lambda x: x[1], reverse=True)  
  
for doc, sim in resultados:  
    print(f" {sim:.4f}  '{doc}'")
```

```
Recursos baixados!  
Busca semântica para: inteligência artificial aprende  
  
0.2695  'machine learning processa dados'  
0.1069  'rede neural treina modelo'  
-0.2988  'gato dorme no sofá'
```


Fórmulas essenciais

Similaridade de cosseno

$$\cos(A,B) = \frac{A \cdot B}{|A| \times |B|}$$

Produto escalar ($A \cdot B$)

$$A \cdot B = A[0]*B[0] + A[1]*B[1] + \dots + A[n]*B[n]$$

Norma de um vetor ($|A|$)

$$|A| = \sqrt{A[0]^2 + A[1]^2 + \dots + A[n]^2}$$

Analogia vetorial

$$\text{resultado} = \text{vetor}(A) - \text{vetor}(B) + \text{vetor}(C)$$

$$\text{ex: } \text{vetor}(\text{"rei"}) - \text{vetor}(\text{"homem"}) + \text{vetor}(\text{"mulher"}) \approx \text{vetor}(\text{"rainha"})$$

Vetor médio de uma frase

$$\text{vetor_frase} = (v1 + v2 + \dots + vn) / n$$

Comparativo das representações

Representação	Dimensões	Semântica	Ordem	Contexto
BoW	V (10k–100k)	✗	✗	✗
TF-IDF	V (10k–100k)	✗	✗	✗
N-gramas	V^2 (enorme)	✗	Parcial	✗
Word2Vec / GloVe	50–300	✓	✗	✗
BERT	768	✓	✓	✓

Próximas Aulas

- **Semana 07 — Representações conceituais:** WordNet e grafos de conhecimento — representação simbólica (regras e ontologias) vs. representação distribuída (embeddings)
- **Semana 10 — Redes Neurais para PLN:** RNN e LSTM que usam embeddings como entrada
- **Semana 11 — Transformers:** BERT e GPT — embeddings contextualizados com atenção global

Referências

CASELI, H. M.; NUNES, M. G. V. (org.). Processamento de linguagem natural: conceitos, técnicas e aplicações em português. São Carlos: BPLN, 2023. Cap. 8, pp. 215–248.

SUMATHI, S.; JANANI, M. Neural Networks for Natural Language Processing. Hershey, PA: Engineering Science Reference, 2020. Cap. 2, pp. 28–65.

SICSÚ, A. L.; SAMARTINI, A.; BARTH, N. L. Técnicas de Machine Learning. São Paulo: Editora Blucher, 2023. Cap. 5, pp. 89–118.

Recursos online gratuitos

- Word2Vec original (Mikolov et al., 2013): <http://arxiv.org/abs/1301.3781>
- GloVe (Pennington et al., 2014): <http://nlp.stanford.edu/projects/glove/>
- Gensim Word2Vec: <http://radimrehurek.com/gensim/models/word2vec.html>
- NILC — Embeddings para português: <http://nilc.icmc.usp.br/embeddings>

Alternativa: <https://huggingface.co/nilc-nlp>

- Embedding Projector (Google): <http://projector.tensorflow.org> — visualização interativa de embeddings



IBMEC.BR

 /IBMEC

 IBMEC

 @IBMEC_OFICIAL

 @IBMEC

 **ibmec**